



JANUARY 15, 2026

ARTIFICIAL INTELLIGENCE

DR. IRUM MATLOOB

ASSIGNMENT-3

HAJRA SARWAR
2023-BSE(5A)-022



TABLE OF CONTENTS

TASK 1: AI RAG System Implementation (Practical Component)	3
1. Environment Setup.....	3
2. Project Structure Initialization.....	4
3. Installing Dependencies.....	5
4. Setting Up OpenAI API Key.....	6
5. Vector Database Setup (Quadrant).....	7
6. Creating Vector Database Client.....	8
7. Data Loader Implementation.....	9
8. FastAPI Server Setup.....	10
9. Inngest Integration.....	11
10. PDF Ingestion Function.....	12
11. Query Function Implementation.....	13
12. Streamlit Frontend Creation.....	14
13. Testing and Final Output.....	15
 TASK 2: Conceptual Understanding (Theory Component)	 16
1. What is Retrieval-Augmented Generation (RAG)?.....	16
2. Difference Between RAG and Agentic RAG.....	17
3. Purpose of RAG.....	18
4. Benefits of Using RAG in Agentic Development.....	19
 TASK 3: AI RAG Development Explanation	 20
1. System Architecture.....	20
2. Component Description.....	21
3. Data Flow Process.....	22

AI RETRIEVAL-AUGMENTED GENERATION SYSTEM

TASK 1:

AI RAG SYSTEM IMPLEMENTATION

Steps:

1. Environment Setup

The development environment was prepared according to the video tutorial to ensure compatibility with a production-ready RAG system.

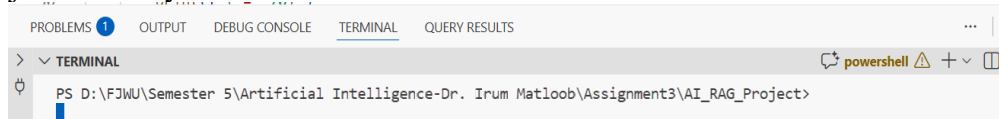
Steps:

- Installed **Python 3.10**
- Enabled *Add Python to PATH*
- Verified installation using:

`python --version`

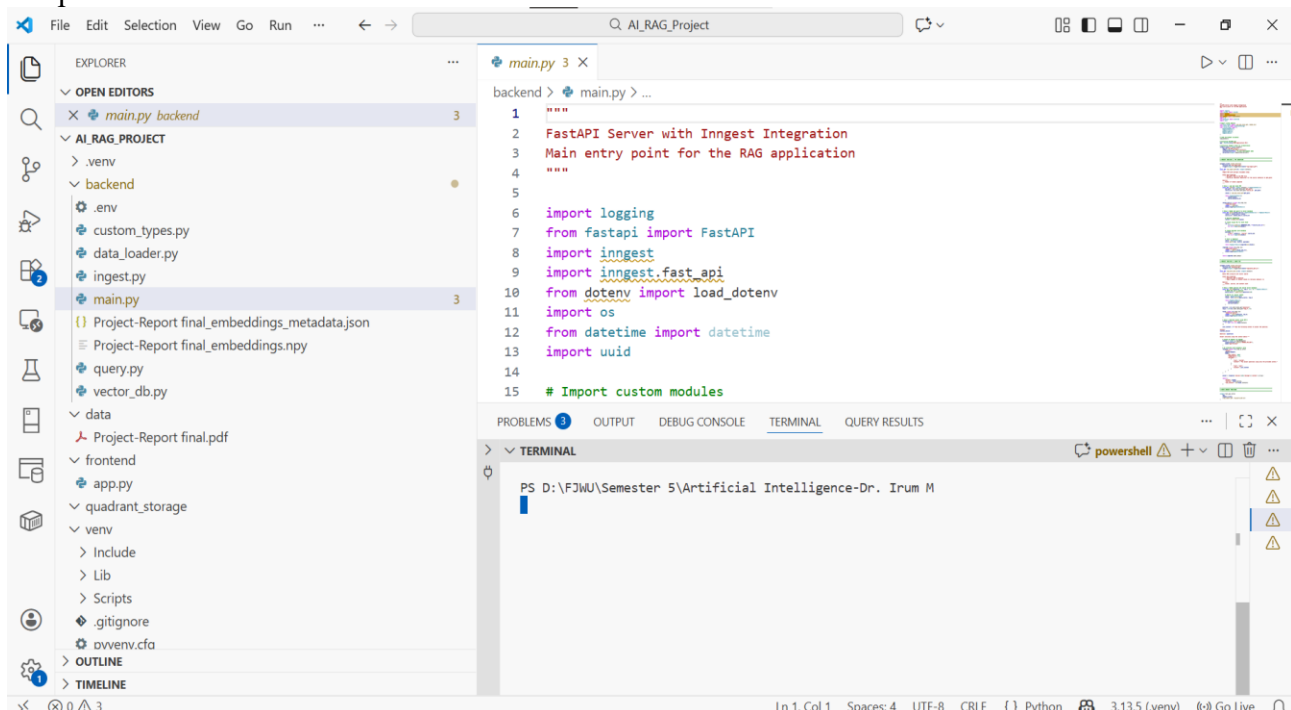
```
PS C:\Users\HP> python --version
Python 3.13.5
PS C:\Users\HP>
```

Created project directory:



2. Project Structure Initialization

A modular, production-level project structure was created to separate backend, frontend, and data components.



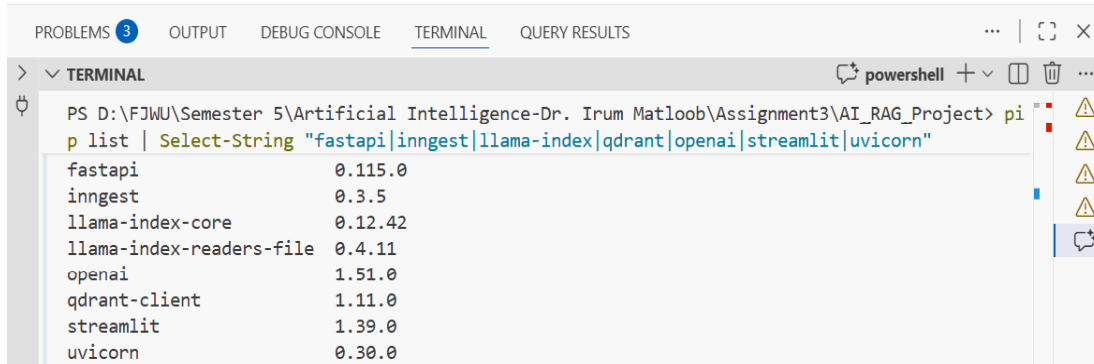
3. Installing Dependencies

Python dependencies were installed using pip.

Key Libraries

- FastAPI
- Inngest
- LlamaIndex

- Qdrant Client
- OpenAI
- Streamlit

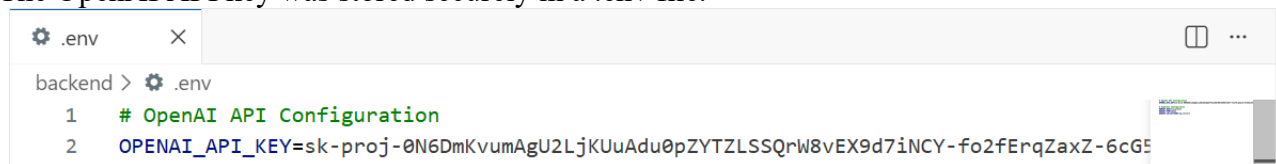


```

PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Assignment3\AI_RAG_Project> pip list | Select-String "fastapi|inngest|llama-index|qdrant|openai|streamlit|uvicorn"
fastapi              0.115.0
inngest              0.3.5
llama-index-core     0.12.42
llama-index-readers-file 0.4.11
openai               1.51.0
qdrant-client        1.11.0
streamlit            1.39.0
uvicorn              0.30.0
  
```

4. Setting Up OpenAI API Key

The OpenAI API key was stored securely in a .env file.



```

# OpenAI API Configuration
OPENAI_API_KEY=sk-proj-0N6DmKvumAgU2LjKUuAdu0pZYTZLSSQrW8vEX9d7iNCY-fo2fErqZaxZ-6cG5
  
```

- .env added to .gitignore
- Prevents accidental exposure

5. Vector Database Setup (Qdrant)

Qdrant was used as the vector database for semantic search.

Docker Command

```
docker run -d --name qdrant_rag_db -p 6333:6333 \
-v "%cd%/qdrant_storage:/qdrant/storage" qdrant/qdrant
```

Verified at:

<http://localhost:6333/dashboard>

 Screenshots 6–7: Docker & Qdrant dashboard

6. Creating Vector Database Client

A dedicated client module was created to manage vector operations.

Features

- Auto collection creation
- Cosine similarity
- 3072-dimension vectors
- Metadata storage

 Screenshot 8: vector_db.py

7. Data Loader Implementation

PDF loading, chunking, and embedding generation were handled using LlamaIndex and OpenAI.

Chunking Strategy

- Chunk size: 1000
- Overlap: 200
- Sentence-based splitting

 Screenshot 9: data_loader.py

8. FastAPI Server Setup

FastAPI was configured as the backend server and integrated with Inngest.

Key Points

- Central entry point (main.py)
- Inngest endpoint auto-generated
- Local development mode enabled

 *Screenshot 10: FastAPI setup*

9. Inngest Integration

Inngest was used for orchestration and observability.

Commands

```
npx inngest-cli@latest dev -u http://127.0.0.1:8000/api/inngest --no-discovery
```

```
uvicorn main:app --reload
```

Dashboard: <http://localhost:8288>

 *Screenshots 11–12: Inngest dashboard*

10. PDF Ingestion Function

PDF ingestion was implemented as an Inngest function with observable steps.

Steps

1. Load and chunk PDF
2. Embed chunks
3. Store vectors in Qdrant

 *Screenshot 13: Ingestion function*

11. Query Function Implementation

The query function retrieves relevant chunks and generates answers using GPT-4o-mini.

Process

- Embed query
- Vector search
- Context-based generation
- Source attribution

 *Screenshots 14–15: Query execution*

12. Streamlit Frontend Creation

A Streamlit UI was created for PDF upload and Q&A.

Features

- PDF upload
- Interactive chat
- Source display

 *Screenshot 16: Streamlit UI*

13. Testing and Final Output

All components were tested end-to-end.

Verified

- PDF ingestion
- Vector storage
- Accurate answers
- Retry mechanism

 *Screenshots 17–23: Testing results*

1. What is Retrieval-Augmented Generation (RAG)?

RAG combines **information retrieval** with **language generation** to produce accurate, source-grounded answers.

Three Phases

1. Retrieval
2. Augmentation
3. Generation

Why RAG?

- Reduces hallucinations
- Enables up-to-date knowledge
- Supports source citation

2. Difference Between RAG and Agentic RAG

Aspect	Traditional RAG	Agentic RAG
Workflow	Fixed	Adaptive
Reasoning	Single-step	Multi-step
Tool Use	Limited	Multiple tools
Autonomy	Low	High

Workflow	Fixed	Adaptive
Reasoning	Single-step	Multi-step
Tool Use	Limited	Multiple tools
Autonomy	Low	High

3. Purpose of RAG

- Overcomes LLM knowledge cutoff
- Improves factual accuracy
- Enables domain-specific knowledge
- Allows source verification

4. Benefits of Using RAG in Agentic Development

- Evidence-based decisions
 - Iterative retrieval
 - Autonomous problem solving
 - Enterprise scalability
-

TASK 3:

AI RAG DEVELOPMENT EXPLANATION

1. System Architecture

The system follows a **production-ready, modular architecture** integrating frontend, backend, orchestration, vector storage, and LLM services.

 *Architecture Diagram*

2. Component Description

Key Components

- PDF Documents (Knowledge Base)
- OpenAI Embedding Model
- Qdrant Vector Database
- Retriever
- GPT-4o-mini
- Inngest Orchestration
- Streamlit Frontend

Each component plays a distinct role in ensuring accuracy, scalability, and reliability.

3. Data Flow Process

Ingestion Flow

PDF → Chunking → Embedding → Vector Storage

Query Flow

Question → Retrieval → Context → LLM → Answer

Error Handling

- Automatic retries
- Logging
- Observability via dashboard

CONCLUSION

This assignment successfully demonstrates a **production-ready AI RAG system** with:

- Complete ingestion-to-generation pipeline
- Reliable orchestration using Inngest
- Accurate semantic search with Qdrant
- Grounded answers using GPT-4
- Clean and user-friendly frontend

The project reflects **strong practical implementation** and **clear theoretical understanding** of modern RAG systems.

REFERENCES

1. *How to Build a Production-Ready RAG AI Agent in Python* – YouTube
2. Inngest Documentation
3. Qdrant Documentation
4. OpenAI API Documentation